

Сравнение и анализ оптимизаций компиляторов

Шурыгин Ярослав Вячеславович

Группа: 25.Б42

Кафедра: системного программирования
Научный руководитель: Гориховский Вячеслав Игоревич
Номер семестра практики: 2

1. Введение

Компиляторы языка C выполняют многие оптимизации, но делают это по-разному: при одинаковых флагах сборки размер и быстродействие генерируемого кода могут заметно различаться. Понимание этого поведения полезно при разработке на C: оно позволяет выбирать компилятор под конкретную задачу.

В данной работе проводится сравнение и анализ оптимизаций, выполняемых компиляторами языка C: GCC, Clang и Intel ICX. В качестве прикладной задачи выбрана обработка изображений с использованием библиотеки `libgd`¹ версии 2.3.3. Сравнение проводится на двух наборах исходного кода: на тесте оптимизаций `optbench.c`² и на программе `libgd_test.c`³, использующей библиотеку `libgd`¹ версии 2.3.3. В рамках анализа выполняются измерения времени выполнения программ и размеров исполняемых файлов, а также проводится сравнение сгенерированного ассемблерного кода.

2. Постановка задачи

Цель работы — исследовать и сравнить результаты оптимизации программ на языке C, выполненной компиляторами GCC, Clang и ICX.

Для достижения цели требуется решить следующие задачи:

1. Выбрать компиляторы и обосновать их выбор.
2. Исследовать оптимизации на программе `optbench.c`.
3. Сравнить ассемблерный код, сгенерированный без оптимизации и с применением оптимизации.
4. Сравнить размер исполняемых файлов и время выполнения.
5. Сопоставить способы оптимизации между компиляторами.
6. Провести аналогичный эксперимент на программе обработки изображений с использованием `libgd` и функции `gdImageGaussianBlur`.
7. Сформулировать выводы по результатам экспериментов.

3. Обзор

Для исследования выбраны GCC, Clang и ICX.

GCC 13.3.0 выбран в качестве основного компилятора, поскольку он широко распространён и поддерживает большое количество архитектур.

Clang 18.1.3 позволяет сравнить GCC с другим компилятором, использующим инфраструктуру LLVM.

ICX 2026.0.0 включён в исследование для оценки оптимизаций компилятора Intel на процессоре AMD.

В качестве тестовой задачи для сравнения возможностей оптимизаторов компиляторов языка C был выбран алгоритм гауссового размытия изображений, выполняющий свёртку двумерной матрицы пикселей с ядром 3×3 . Данный тест представляет интерес для анализа оптимизаций, поскольку содержит вложенные циклы, операции умножения-сложения, работает с матрицами.

4. Описание предлагаемого решения

Эксперимент проводится на системе:

- архитектура: AMD64 (x86-64);
- операционная система: Ubuntu 24.04.3 LTS;
- процессор: AMD Ryzen 7 8845H.

Для программы `optbench.c` рассматриваются варианты без оптимизации (`00`), с оптимизацией производительности (`02`) и с оптимизацией размера (`0s`). Для анализа сгенерированного кода используются ассемблерные листинги.

Дополнительный эксперимент проводится на собственной программе на языке C, использующей библиотеку `libgd` и функцию `gdImageGaussianBlur`. Библиотека связывается статически с программой, после чего полученный исполняемый файл дизассемблируется.

Измерения времени выполнения и размеров исполняемых файлов выполняются для вариантов, собранных с флагами `00`, `02` и `0s`.

Сравнение дизассемблера — для `00`, `02`

¹Исходный код библиотеки `libgd` версии 2.3.3: <https://github.com/libgd/libgd/tree/gd-2.3.3> (дата обращения: 22.09.2026).

²Исходный код теста `optbench.c`: https://github.com/stuffacc/compiler_cmp/blob/master/optbench/src/optbench.c (дата обращения: 22.09.2026).

³Исходный код теста `libgd_test.c`: https://github.com/stuffacc/compiler_cmp/blob/master/libgd_test/src/libgd_test.c (дата обращения: 22.09.2026).

5. Эксперименты

Размеры секций исполняемых файлов измерялись с помощью утилиты `GNU size` версии 2.42. Для замера общего размера исполняемого файла на диске применялась утилита `stat` версии 9.4. Оценивались размеры следующих секций исполняемого файла: `.text`, `.data` и `.bss`. Замеры времени выполнения проводились с использованием утилиты `hyperfine` версии 1.18.0. Перед измерениями каждый исполняемый файл выполнялся 100 раз в режиме прогрева, после чего проводилось 1000 измерительных запусков.

5.1. Анализ производительности

- **optbench.c⁴**: Все три исследованных компилятора (`gcc`, `clang`, `icx`) демонстрируют **практически одинаковое время выполнения** программы. Поскольку разброс измерений соизмерим с разницей между средними значениями любых двух конфигураций, утверждать о статистически значимом превосходстве одного компилятора или уровня оптимизации над другим **нельзя**.
- **libgd_test.c⁵**: По сравнению с уровнем оптимизации `00`, оптимизация `02` дает заметный прирост быстродействия у всех компиляторов (от 18 до 29% быстрее). При этом результаты `02` у всех трех компиляторов практически одинаковы: от 72,1 до 73,1 мс. Разница между ними составляет около 1 мс и сопоставима с погрешностью измерений, поэтому говорить о превосходстве GCC, Clang или ICX по быстродействию на уровне `02` некорректно.

5.2. Анализ размера бинарного файла

- **optbench.c⁴**: Общий размер исполняемого файла на диске (замеренный с помощью `stat`) остается практически неизменным во всех конфигурациях: примерно 17 КБ. Однако оптимизация по размеру (`0s`) дает выраженный и ожидаемый эффект для секции `.text icx 0s`: 2629 байт — **наименьший размер кода среди всех конфигураций**. Экономия составляет примерно 24% по сравнению с `icx 00` (3443 байт). Таким образом, компилятор `icx` с флагом `0s` генерирует наиболее компактный машинный код для данного теста, тогда как результаты GCC и Clang близки друг к другу: 3021 и 3029 байт соответственно.
- **libgd_test.c⁵**: Оптимизация по размеру (`0s`) заметно уменьшает секцию `.text` у `gcc` и `clang`. Наименьший размер секции `.text` показывает конфигурация GCC с флагом `0s`: 59 322 байта, что на 33,6% меньше, чем у GCC с флагом `00`. Для `clang-0s` уменьшение составляет 31,4% относительно `clang-00`.

Анализ дизассемблированного кода показывает, что `icx` использует собственные функции Intel. Эти функции реализуют служебные и математические операции и занимают дополнительное место в итоговом исполняемом файле.

Поэтому размер секции `.text` у программ, собранных с помощью `icx`, значительно больше, чем у версий, собранных с помощью `gcc` и `clang`.

5.3. Анализ ассемблера/дизассемблера

- **optbench.c⁶**: Все компиляторы (GCC, Clang, ICX) на уровнях `02/0s` выполняют базовый набор оптимизаций: свёртку констант, удаление мёртвого кода и лишних присваиваний, удаление общих подвыражений, снижение мощности операции (умножение заменяется сдвигом), сокращение числа обращений к памяти. Простые циклы с константным количеством итераций могут полностью удаляться, если результат цикла можно вычислить на этапе компиляции, а циклы со счётчиком заменяются циклами с адресной арифметикой.

ICX (Intel) — полностью разворачивает малые циклы (`loop_jamming`, `loop_unrolling`), эффективнее использует известные значения переменных и в ряде случаев заменяет обращения к памяти непосредственными константами, деление заменяет умножением со сдвигом, выносит инвариантный код из циклов. На `0s`, наоборот, не разворачивает циклы и даёт самый компактный код. На `02` появляется недостижимый блок кода.

Только Clang применяет SSE-векторизацию при записи элементов массива в рассматриваемом тесте. Однако в данном тесте векторизованный вариант не приводит к заметному ускорению и увеличивает размер кода.

GCC генерирует компактный, предсказуемый код, а показатели его производительности сопоставимы с результатами Clang и ICX.

- **libgd_test.c⁷**: Все компиляторы (GCC, Clang, ICX) на оптимизированном уровне `02` уменьшают количество обращений к памяти: устраняются избыточные операции загрузки и сохранения.

Clang и ICX разворачивают вложенный `i`-цикл, из-за чего код становится длиннее, но уменьшается число проверок и переходов. GCC в этой функции циклы не разворачивает.

⁴Результаты измерений производительности и размера бинарного файла для теста `optbench.c`: https://github.com/stuffacc/compiler_cmp/blob/master/optbench/tables/optbench_statistic.csv (дата обращения: 22.09.2026).

⁵Результаты измерений производительности и размера бинарного файла для теста `libgd_test.c`: https://github.com/stuffacc/compiler_cmp/blob/master/libgd_test/tables/libgd_test_statistic.csv (дата обращения: 22.09.2026).

⁶Таблицы сравнений листингов ассемблерного кода для теста `optbench.c`: https://github.com/stuffacc/compiler_cmp/tree/master/optbench/tables (дата обращения: 22.09.2026).

⁷Таблицы сравнений дизассемблированных файлов для теста `libgd_test.c`: https://github.com/stuffacc/compiler_cmp/tree/master/libgd_test/tables (дата обращения: 22.09.2026).

ISX дополнительно заменяет деление умножением ($1/\text{filter_div}$ вычисляется один раз до циклов и далее используется `mul` вместо `div`) — это дешевле деления.

В рассматриваемом фрагменте ни один из исследованных компиляторов не выполняет векторизацию, вычисления остаются скалярными.

6. Заключение

В ходе выполнения работы получены следующие результаты:

1. Осуществлён выбор компиляторов и приведено обоснование их использования.
2. Проведено исследование влияния уровней оптимизации на работу программы `optbench.c`.
3. Выполнен сравнительный анализ ассемблерного кода, сгенерированного без оптимизации и с применением оптимизаций.
4. Проведено сравнение размеров исполняемых файлов и времени выполнения программ.
5. Выполнено сопоставление методов оптимизации, используемых различными компиляторами.
6. Проведено аналогичное исследование программы обработки изображений с использованием библиотеки `libgd` и функции `gdImageGaussianBlur`.
7. Сформулированы выводы по результатам проведённых исследований.

Материалы, иллюстрирующие выполнение работы, размещены по адресу: https://github.com/stuffacc/compiler_